

Security Review

Cantina Managed

COINBASE: CONTRACTS DYNAMIC UP- GRADES

Cantina Managed review by:

Sujith S, Lead Security Researcher

Cryptara, Security Researcher



September 19, 2026



Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Truncated initial schedules make the node and verifier commit to different histories	4
3.2	Low Risk	5
3.2.1	Clearing an upgrade under a scheduled successor permanently holes the id	5
3.2.2	Asymmetric timestamp checks prevent simultaneous upgrades from being delayed together	6
3.2.3	Imported schedules leave the node-required minimum protocol version unset . . .	7
3.2.4	Initialization can make a near-term activation immutable before nodes observe it .	8
3.2.5	A permitted emergency schedule change can reach nodes after the old upgrade has activated	9
3.3	Informational	10
3.3.1	NatSpec claims L1-origin pinning and journal domain fields <code>AggregateVerifier</code> does not implement	10
3.3.2	<code>AggregateVerifier</code> comment still describes freeze-at-activation semantics . . .	10
3.3.3	Initialization proof L1 origin is unbound from the factory <code>11Head</code>	11



1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).



2 Security Review Summary

Base is a secure, low-cost, builder-friendly Ethereum L2 built to bring the next billion users onchain.

From Aug 19th to Aug 21st the Cantina team conducted a review of [contracts](#) on commit hash [4f7acda5](#). The team identified a total of **9** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	1	0	1
Low Risk	5	5	0
Gas Optimizations	0	0	0
Informational	3	3	0
Total	9	8	1

2.1 Scope

The security review had the following components in scope for [contracts](#) on commit hash [4f7acda5](#):

```
src/L1
├── ProtocolVersions.sol
├── proofs
│   └── AggregateVerifier.sol
```



3 Findings

3.1 Medium Risk

3.1.1 Truncated initial schedules make the node and verifier commit to different histories

Severity: Medium Risk.

Context: `ProtocolVersions.sol`#L149.

Description: The initial schedule is a positional cross-system commitment, but neither `Deploy Config` nor `ProtocolVersions.initialize()` checks that the array contains an entry for every contract-backed upgrade known to the node. A proper prefix is therefore accepted as a complete schedule and permanently registers only that prefix onchain.

The two sides interpret the missing suffix differently. `ProtocolVersions.activatedScheduleId()` can only hash entries present in `_timestamps`, whereas the node zips the supplied prefix onto its existing `RollupConfig`. Omitted upgrades receive no explicit `Never` override, so their static activation timestamps remain effective. `ScheduleId::pin()` subsequently walks all thirteen `BaseUpgrade::CONTRACT_VARIANTS` and includes any active static suffix. Once an omitted fork is active, the verifier pins one schedule ID while honest TEE and ZK provers derive another. Because `scheduleId` is part of both proof journals, every honest proof is rejected and aggregate games cannot make progress. Setting a valid `minimumProtocolVersion` fixes the separate startup gate but does not repair this commitment mismatch.

Proof of Concept: The first four entries below are a correctly ordered Base mainnet prefix, but the later known entries through Beryl are omitted. Add this test to `ProtocolVersions_ActivatedScheduleId_Test` in `test/L1/ProtocolVersions.t.sol`:

```
function test_truncatedInitialSchedule_divergesFromNodeHistory() external {
    uint64[] memory prefix = new uint64[](4);
    prefix[0] = 1_686_789_347; // Regolith, normalized to the L2 genesis timestamp.
    prefix[1] = 1_704_992_401; // Canyon.
    prefix[2] = 1_708_560_000; // Delta.
    prefix[3] = 1_710_374_401; // Ecotone.

    IProtocolVersions truncated = _importSchedule(prefix);

    bytes32 contractScheduleId =
        ↪ truncated.activatedScheduleId(BASE_MAINNET_BERYL_TIMESTAMP);
    bytes32 nodeScheduleId =
        0xadd4aa9bd3532969035a9543c16b8c7d71298e15836f0ac731fdd3eea552c6e2;

    // The contract stops after Ecotone (id 3).
    assertEq(
        contractScheduleId,
        0xd694b06480e4f65f86d9d2a7be6a0d34149ef0bd0eae338f3a26d769911bc0a8
    );

    // The node retains its static Fjord..Beryl suffix and hashes through id 11.
    assertNotEq(contractScheduleId, nodeScheduleId);
}
```

The `nodeScheduleId` value is the existing cross-implementation Base mainnet golden produced by `ScheduleId::pin()` at Beryl. `AggregateVerifier.initializeWithInitData()` stores `contractScheduleId`, while the proof boot path journals `nodeScheduleId`; the final journal hashes therefore differ even when every other proof input is identical.

Recommendation: For migrations, generate the imported array from the node's complete effective contract-backed schedule and require explicit zero entries for known but unscheduled upgrades. Reject nonempty proper prefixes, verify the resulting onchain schedule ID against a node-generated



value before enabling the game type, and make nodes reject incomplete authoritative arrays instead of silently falling back to static suffix values.

Coinbase: The deployment of `ProtocolVersions` will happen in our contract-deployments repo and will have its own security review. I don't think we need a base/contracts change for this.

Cantina Managed: Acknowledged.

3.2 Low Risk

3.2.1 Clearing an upgrade under a scheduled successor permanently holes the id

Severity: Low Risk.

Context: `ProtocolVersions.sol#L227-L242`, `ProtocolVersions.sol#L372-L376`.

Description: The function `setTimestamp()` forbids filling a zero-valued hole once a later upgrade is already scheduled, but clearing an earlier upgrade to zero is unrestricted:

```
function setTimestamp(uint256 id, uint64 timestamp) external {
    // ...
    if (timestamp != 0) {
        if (current == 0) _assertNoScheduledSuccessor(id);
        _assertTimestampAfterPrevious(id, timestamp);
        _assertTimestampBeforeNext(id, timestamp);
    }
    _writeTimestamp(id, timestamp);
}
```

`_assertNoScheduledSuccessor` therefore runs only on the `0 → nonzero` path. Passing `timestamp == 0` always proceeds to `_writeTimestamp`, which refreshes the hash chain from that id onward.

After clearing upgrade `A` while successor `B` remains scheduled:

1. `scheduleId` / `activatedScheduleId` for affected cutoffs change and embed `(id_A, 0)` in the prefix.
2. Re-scheduling `A` reverts `ProtocolVersions_StaticScheduleHole` until every higher scheduled successor is cleared first.

This contradicts the NatSpec on `activatedScheduleId`, which states that zero-valued holes below a scheduled successor cannot be scheduled later in order to prevent inactive prefix holes from moving activated schedule commitments. The fill-side guard enforces half of that claim; the clear-side path falsifies it.

Under `FREEZE_WINDOW` and `AggregateVerifier`'s `L2TimestampInFuture` check, an upgrade that could appear in a creatable proof-game pin is already frozen, so this does not create divergent pins across creatable `AggregateVerifier` games. The residual issue is an owner-gated operational foot-gun with a permanent recovery cost until successors are cleared, plus the documentation mismatch.

Proof of Concept:

```
function test_clearUnderSuccessor_changesActivatedPrefixAndFreezesHole() public {
    uint64 l1Now = 3_000_000;
    vm.warp(l1Now);

    uint64 t0 = l1Now + protocolVersions.MIN_NOTICE() + 100;
    uint64 t1 = l1Now + protocolVersions.MIN_NOTICE() + 200;

    vm.startPrank(owner);
    protocolVersions.registerUpgrade(t0, 0);
    protocolVersions.registerUpgrade(t1, 0);
    vm.stopPrank();
}
```



```

bytes32 beforeClear = protocolVersions.activatedScheduleId(t1);
bytes32 link0 = keccak256(abi.encode(bytes32(0), uint256(0), t0));
bytes32 link1 = keccak256(abi.encode(link0, uint256(1), t1));
assertEq(beforeClear, link1);

vm.prank(owner);
protocolVersions.setTimestamp(0, 0);

bytes32 hole0 = keccak256(abi.encode(bytes32(0), uint256(0), uint64(0)));
bytes32 link1Hole = keccak256(abi.encode(hole0, uint256(1), t1));
assertEq(protocolVersions.activatedScheduleId(t1), link1Hole);
assertNotEq(beforeClear, link1Hole);

uint64 refill = 11Now + protocolVersions.MIN_NOTICE() + 150;
vm.expectRevert(
    abi.encodeWithSelector(ProtocolVersions.ProtocolVersions_StaticScheduleHole
        ↪ .selector, 0,
        ↪ 1)
);
vm.prank(owner);
protocolVersions.setTimestamp(0, refill);
}

```

The test passes.

Recommendation: On clear-to-zero, enforce the same successor check used when filling a hole:

```

if (timestamp == 0) {
    _assertNoScheduledSuccessor(id);
} else {
    if (current == 0) _assertNoScheduledSuccessor(id);
    _assertTimestampAfterPrevious(id, timestamp);
    _assertTimestampBeforeNext(id, timestamp);
}

```

Alternatively, expose an explicit multi-id cancel path that documents recovery, and update the `activatedScheduleId` NatSpec so it no longer claims clear-under-successor is impossible.

Coinbase: Fixed in [PR 415](#).

Cantina Managed: [PR 415](#) resolves the owner-gated footgun and the NatSpec overclaim in `activatedScheduleId`. Cross-game divergence through this path is not possible: clearing is only permitted while $L1 < \text{activation} - \text{FREEZE_WINDOW}$, but an `AggregateVerifier` game against that activation requires $L1 \geq \text{activation}$, so the two windows never overlap; additionally, `GameAlreadyExists` prevents a replacement proposal from being re-created after a clear.

3.2.2 Asymmetric timestamp checks prevent simultaneous upgrades from being delayed together

Severity: Low Risk.

Context: `ProtocolVersions.sol`#[L239](#).

Description: The function `registerUpgrade()` allows consecutive upgrades to share an activation timestamp, but `_assertTimestampBeforeNext()` rejects setting an earlier upgrade to the same timestamp as its successor.

Consequently, the incident responder cannot delay simultaneous upgrades while preserving their atomic activation, requiring owner intervention or split timestamps.

Recommendation: Allow equality in the forward-ordering check and add tests covering equal timestamps in both `setTimestamp()` and `delayTimestamp()`.



Coinbase: Fixed in [PR 419](#).

Cantina Managed: Fix verified.

3.2.3 Imported schedules leave the node-required minimum protocol version unset

Severity: Low Risk.

Context: [ProtocolVersions.sol#L134](#).

Description: `ProtocolVersions.initialize()` imports `_initialSchedule` but never initializes `minimumProtocolVersion`. The standard deployment path calls only this initializer, so deploying an existing chain with any historical activation leaves a positive schedule paired with the default minimum version of zero.

That state is valid to the contract but invalid to the Base node. `AlloyUpgradeSignalReader::map_schedule()` attaches the global minimum version to every imported timestamp, and `UpgradeSignalConfig::validate_signal_has_protocol_version()` rejects every positive timestamp whose version is zero. Startup uses `read_validated_schedule()` before applying either the execution or consensus schedule, so production nodes configured to consume the newly deployed registry fail to start. The runtime refresh path rejects the schedule for the same reason. Recovery requires a separate owner transaction to set the version, leaving an avoidable outage window during migration.

Proof of Concept: Add the following test to `ProtocolVersions_Initialize_Test` in `test/L1/ProtocolVersions.t.sol`:

```
function test_initialize_importLeavesMinimumProtocolVersionUnset() external {
    uint64[] memory schedule = new uint64[](1);
    schedule[0] = 1;

    IProtocolVersions imported = _deployUninitializedProxy();
    vm.prank(EIP1967Helper.getAdmin(address(imported)));
    imported.initialize(_incidentResponder, schedule);

    assertGt(imported.getSchedule()[0], 0);
    assertEq(imported.minimumProtocolVersion(), 0);
}
```

The test passes. The resulting values are then mapped by the node into an `UpgradeSignal` with `activation_timestamp = 1` and `protocol_version = 0`, which deterministically hits the following startup validation:

```
if signal.activation_timestamp > 0 && signal.protocol_version == U256::ZERO {
    return Err(UpgradeSignalError::missing_protocol_version(
        signal.upgrade_id.contract_id().to_string(),
    ));
}
```

`SystemDeploy._initializeOPChain()` does not make a subsequent `setMinimumProtocolVersion()` call, so this is also the final state produced by the standard deployment flow.

Recommendation: Add an initial minimum protocol version to the initializer and deployment configuration, and set it atomically with the imported schedule. Initialization should revert when any imported timestamp is nonzero and the supplied minimum version is zero; the deployment script should also assert the post-deployment schedule and minimum version before completing.

Coinbase: Fixed in [PR-420](#) and [PR-435](#).

Cantina Managed: Verified fix. PR-420 did not fully mitigate the finding, as zero minimum protocol versions remained possible through alternative code paths. Following our request, PR-435 added the necessary safeguards and fully resolves the issue.



3.2.4 Initialization can make a near-term activation immutable before nodes observe it

Severity: Low Risk.

Context: `ProtocolVersions.sol#L124-L128`.

Description: `initialize()` intentionally bypasses `MIN_NOTICE` so historical activations can be imported, but it applies the exemption to every timestamp, including future ones. A deployment can therefore import an activation at or inside `block.timestamp + FREEZE_WINDOW`. The timestamp is accepted and immediately becomes impossible for either the owner or incident responder to clear or delay because all later mutations call `_assertNotFrozen()`.

Nodes read the registry at the finalized L1 head by default. A near-term activation can consequently take effect before the initialization transaction is finalized and observed by the fleet, leaving running nodes on the old rules while newly started nodes apply the imported schedule. Besides a consensus split or outage at the activation boundary, retroactive application can miss fork-specific one-time transition transactions. The emergency role cannot recover the network because the timestamp was frozen at the moment it was created.

Proof of Concept: Add the following test to `ProtocolVersions_Initialize_Test` in `test/L1/ProtocolVersions.t.sol`:

```
function test_initialize_acceptsAlreadyFrozenFutureActivation() external {
    vm.warp(1_800_000_000);

    uint64 activation = uint64(block.timestamp) + protocolVersions.FREEZE_WINDOW();
    uint64[] memory schedule = new uint64[](1);
    schedule[0] = activation;

    IProtocolVersions imported = _deployUninitializedProxy();
    vm.prank(EIP1967Helper.getAdmin(address(imported)));
    imported.initialize(_incidentResponder, schedule);

    assertEq(imported.getSchedule()[0], activation);

    vm.expectRevert(
        abi.encodeWithSelector(
            IProtocolVersions.ProtocolVersions_ActivationFrozen.selector,
            uint256(0),
            activation
        )
    );
    vm.prank(_incidentResponder);
    imported.delayTimestamp(0, activation + imported.MIN_NOTICE());

    vm.expectRevert(
        abi.encodeWithSelector(
            IProtocolVersions.ProtocolVersions_ActivationFrozen.selector,
            uint256(0),
            activation
        )
    );
    vm.prank(_owner);
    imported.setTimestamp(0, 0);
}
```

The initializer accepts the future activation, and both recovery paths revert in the same block.

Recommendation: Keep the initializer exemption only for historical timestamps. Any nonzero imported timestamp greater than `block.timestamp` should satisfy the same `MIN_NOTICE` requirement as `registerUpgrade()` and `setTimestamp()`; alternatively, require future upgrades to be registered only after initialization.



Coinbase: Fixed in [PR 421](#).

Cantina Managed: Fix verified.

3.2.5 A permitted emergency schedule change can reach nodes after the old upgrade has activated

Severity: Low Risk.

Context: [ProtocolVersions.sol#L368](#).

Description: `ProtocolVersions` allows an upgrade scheduled for `A` to be cleared or delayed at `A - 1801`. A `RuntimeAdmin` node using the default `Finalized` block tag may take a nominal 27 minutes 48 seconds to observe the change: 12 minutes 48 seconds for finality plus up to 15 minutes for its next poll.

Under that nominal timing, the mutation is observed at approximately `A - 133`, so the boundary alone does not prove late application. The issue occurs if permitted timestamp drift has already taken the L2 head past `A`, or if finality or delivery exceeds the nominal estimate. Upgrade signalling defaults to `MetricsOnly`, so only nodes explicitly configured to apply runtime updates are affected.

Once observed, the node applies the amended schedule without comparing it to its current L2 head. This does not immediately rewrite stored blocks, but later execution, replay, derivation, reorg handling, synchronization, or proving may interpret affected blocks under different fork rules and reject them or halt.

Proof of Concept: The existing test confirms that the last second before the freeze window remains mutable:

```
uint64 activation = _scheduleCanyon(100);

vm.warp(activation - protocolVersions.FREEZE_WINDOW() - 1);
vm.prank(_owner);
protocolVersions.setTimestamp(CANYON, 0);

assertEq(protocolVersions.getSchedule()[CANYON], 0);
```

Run it with:

```
forge test --match-test test_setTimestamp_justBeforeFreezeWindow_succeeds
```

At `A - 1801`, the nominal finalized observation time is `A - 133`. The unsafe case is therefore conditional on the L2 head already being at or beyond `A`, or on observation taking longer than the nominal estimate. `UpgradeSignalRefresher::apply()` and `apply_schedule_to_sink()` receive no current L2-head timestamp and cannot reject such a late change.

Recommendation: At startup and during runtime refresh, reject or safely halt on any schedule change that would alter fork rules at or before the current L2 head. Also use a schedule-freshness gate or a reorg-aware safe/latest read near activation; increasing the Solidity freeze window alone cannot account for unbounded L1 finality delays.

Coinbase: Fixed in [PR-424](#). Furthermore, the sequencers can be updated to use safe head polling as a runtime config.

Cantina Managed: Verified fix. PR-424 extends the freeze window to one hour, while sequencers will use safe-head polling through runtime configuration. Together, these measures address the late-application risk, no additional code changes are required.



3.3 Informational

3.3.1 NatSpec claims L1-origin pinning and journal domain fields `AggregateVerifier` does not implement

Severity: Informational.

Context: `AggregateVerifier.sol#L962-L975`, `ProtocolVersions.sol#L33-L36`.

Description: `ProtocolVersions` documents that proof journals bind to the schedule in effect at the game's L1 origin and that cross-chain domain separation comes from committing the L2 chain id and registry address alongside `scheduleId` :

```
/// Proof journals bind to `scheduleId`, pinning every proof in a
/// dispute game to the schedule in effect at the game's L1 origin block;
↪ cross-chain domain
/// separation is provided by the journal itself, which commits the L2 chain id and
↪ registry
/// address alongside `scheduleId`.
```

`AggregateVerifier.initializeWithInitData` instead pins via a linear L2 claim timestamp derived from genesis and block time, independent of the verified L1 origin and of the factory `l1Head` . The TEE/ZK journals pack `CONFIG_HASH` , the image or range hash, and `scheduleId` only:

```
bytes32 journal = keccak256(
    abi.encodePacked(
        proposer,
        l1OriginHash,
        startingRoot,
        startingL2SequenceNumber,
        endingRoot,
        endingL2SequenceNumber,
        intermediateRoots,
        CONFIG_HASH,
        TEE_IMAGE_HASH, // or ZK_RANGE_HASH
        scheduleId
    )
);
```

The `L2_CHAIN_ID` and `PROTOCOL_VERSIONS` immutables are set in the constructor but never appear in the journal encoding. Offchain, Rust, or TEE builders that follow the NatSpec may bind the wrong schedule view or omit domain fields that the onchain verifier never checks. Domain separation then depends entirely on `CONFIG_HASH` composition being unique per deployment.

Recommendation: Align the `ProtocolVersions` NatSpec with `AggregateVerifier` 's pin source and journal layout. Either include `L2_CHAIN_ID` and `address(PROTOCOL_VERSIONS)` in the journal, or document that those values are committed inside `CONFIG_HASH` and enforce that invariant in deploy config.

Coinbase: Fixed in [PR 416](#).

Cantina Managed: Fix verified.

3.3.2 `AggregateVerifier` comment still describes freeze-at-activation semantics

Severity: Informational.

Context: `AggregateVerifier.sol#L430-L433`.

Description: `initializeWithInitData` comments that `ProtocolVersions` only freezes an activation once L1 time reaches it, and that requiring `claimTimestamp <= block.timestamp` keeps the pin canonical:



```
// `ProtocolVersions` only freezes an activation once L1 time reaches it, so a claim
→ whose L2
// timestamp is still in L1's future would pin a schedule the owner can afterwards
→ clear or
// delay. Requiring the claim to have already passed on L1 keeps the pin canonical
→ for life.
if (claimTimestamp > block.timestamp) revert L2TimestampInFuture(claimTimestamp,
→ block.timestamp);
```

After #409, preexisting activations freeze at `activation - FREEZE_WINDOW` via `ProtocolVersions_ActivationFrozen`. The runtime gate is correct; the comment is stale and can mislead reviewers tracing pin/mutation invariants.

Recommendation: Update the comment to state that mutations freeze `FREEZE_WINDOW` before activation, and that the claim-timestamp check additionally ensures the game's L2 time is not still in L1's future.

Coinbase: Fixed in PR 417.

Cantina Managed: Fix verified.

3.3.3 Initialization proof L1 origin is unbound from the factory `l1Head`

Severity: Informational.

Context: `AggregateVerifier.sol#L450-L456`, `AggregateVerifier.sol#L490-L498`.

Description: During `initializeWithInitData`, the contract verifies a proof-supplied L1 origin (`proof[1:65]`) via `_verifyL1Origin`, then journals that origin into the init attestation. The CWIA payload still carries a separate factory `l1Head`, which later `verifyProposalProof` calls journal instead:

```
bytes32 l1OriginHash = bytes32(proof[1:33]);
uint256 l1OriginNumber = uint256(bytes32(proof[33:65]));
_verifyL1Origin(l1OriginHash, l1OriginNumber);

_verifyProof(
    proof[65:],
    proofType,
    gameCreator(),
    l1OriginHash,
    /* ... */
);
```

Nothing requires the init origin to equal `l1Head()`, or to lie within a bounded age of it. A single game can therefore attest one L1 provenance at creation and another in subsequent proofs. This may be intentional, but prover stacks can easily assume a single L1 head binding and produce journals that fail one of the two paths.

Recommendation: Document the split explicitly for prover implementers. If a single provenance is required, enforce `l1OriginHash == l1Head()` (or a bounded relationship) at initialization.

Coinbase: Fixed in PR 418.

Cantina Managed: Fix verified.